CHANDIGARH UNIVERSITY

Discover. Learn. Empower.

PYTHON MINI PROJECT REPORT

PERSONAL EXPENSE DATA ANALYSIS

Using NumPy, Pandas, Matplotlib and Seaborn

Submitted by: Harshpreet Kaur

UID: 25BCD10096

Section: 25BCD 2-A

Subject: Python Programming

Submitted to: Ms. Preeti

Designation: Assistant Professor

Submission Date: 20 October 2026

Project Overview

This project analyses sample personal-expense transactions to identify major spending categories, monthly patterns, typical transaction amounts and payment-method preferences. Pandas is used for data loading, cleaning, grouping and filtering; NumPy calculates statistics; Matplotlib and Seaborn produce charts.

Dataset note: the data is synthetic sample data created for educational demonstration and does not represent real personal financial records.

1. Problem Statement

Expense records can be difficult to understand without summaries. This project uses Python to analyse sample transactions, determine which categories account for the most spending, compare monthly totals, summarize transaction amounts and examine payment methods. It also demonstrates the importance of cleaning data before analysis.

Objectives

- Load and inspect a CSV dataset.
- Clean missing, duplicate, inconsistent and incorrectly typed values.
- Calculate descriptive statistics using NumPy and Pandas.
- Group, aggregate, sort and filter records.
- Examine correlations among numeric variables.
- Create visualizations with Matplotlib and Seaborn.
- Summarize findings and draw a conclusion.

2. Dataset Description

The raw CSV contains 151 rows and 6 original columns. After cleaning, 148 valid transactions remain. The data covers January to June 2025, and a Month field is derived from Date.

Variable	Type	Description
Transaction_ID	Text	Transaction identifier
Date	Date	Date of expense
Category	Categorical text	Food, Shopping, Bills, etc.
Description	Text	Transaction description
Amount	Numeric decimal	Expense amount
Payment_Method	Categorical	Cash, Card or UPI
Month	Derived text	Year-month derived from Date

Code: Load and Inspect the Dataset

```
import pandas as pd
df = pd.read_csv("personal_expenses_raw.csv")
print(df.head())
print("Shape:", df.shape)
print(df.dtypes)
```

Output

```
Raw dataset shape: (151, 6)
Columns: Transaction_ID, Date, Category, Description, Amount, Payment_Method
The code displays the first five rows and the data types.
```

Explanation: read_csv() imports the CSV as a DataFrame, head() previews records, shape reports rows and columns, and dtypes lists each column's current type.

3. Data Cleaning Using Pandas

Cleaning standardizes text and data types and removes records that cannot be analysed reliably. The following code converts dates and amounts, standardizes category and payment labels, fills missing descriptions, removes duplicates and rows missing essential fields, excludes negative amounts and creates Month.

```
df["Date"] = pd.to_datetime(df["Date"], errors="coerce")
df["Amount"] = pd.to_numeric(df["Amount"], errors="coerce")
df["Category"] = df["Category"].astype("string").str.strip().str.title()
df["Payment_Method"] = df["Payment_Method"].astype("string").str.strip().str.upper()
df["Payment_Method"] = df["Payment_Method"].replace(
    {"CASH": "Cash", "CARD": "Card", "UPI": "UPI"})
df["Description"] = df["Description"].fillna("Not specified")
df = df.drop_duplicates()
df = df.dropna(subset=["Date", "Category", "Amount"])
df = df[df["Amount"] >= 0].copy()
df["Month"] = df["Date"].dt.to_period("M").astype(str)
df.to_csv("personal_expenses_cleaned.csv", index=False)
print("Cleaned shape:", df.shape)
print(df.isna().sum())
```

Output

```
Raw rows: 151
Cleaned rows: 148
Rows excluded during cleaning: 3
Missing values in cleaned analytical columns: 0
```

Explanation: errors='coerce' converts invalid values to missing values; strip() removes extra spaces; title() standardizes category labels; drop_duplicates() removes duplicate rows. The cleaned file is saved as a new CSV.

4. Data Analysis Using NumPy and Pandas

4.1 Descriptive Statistics

```
import numpy as np
amounts = df["Amount"].to_numpy()
print("Total:", round(np.sum(amounts), 2))
print("Mean:", round(np.mean(amounts), 2))
print("Median:", round(np.median(amounts), 2))
print("Standard deviation:", round(np.std(amounts, ddof=1), 2))
print("Minimum:", round(np.min(amounts), 2))
print("Maximum:", round(np.max(amounts), 2))
```

Output

```
Number of transactions: 148
Total spending: 142,680.56
Mean: 964.06
Median: 755.28
Sample standard deviation: 733.77
Minimum: 44.48
Maximum: 2,978.27
```

Explanation: the mean is the average, the median is the middle value after sorting, and standard deviation describes variation. A mean above the median suggests that some higher transactions increase the average.

4.2 Grouping and Aggregation

```
category_totals = df.groupby("Category")["Amount"].sum().sort_values(ascending=False)
monthly_totals = df.groupby("Month")["Amount"].sum().sort_index()
payment_counts = df["Payment_Method"].value_counts()
print(category_totals)
print(monthly_totals)
print(payment_counts)
```

Output: Category Totals

```
Shopping: 42,893.49
Bills: 39,537.68
Food: 17,968.66
Entertainment: 16,706.34
Education: 13,890.03
Healthcare: 6,962.33
Transport: 4,722.03
```

Output: Monthly Totals

```
2025-01: 23,140.13
2025-02: 20,698.19
2025-03: 29,309.08
2025-04: 30,579.78
2025-05: 23,080.29
2025-06: 15,873.09
```

Output: Payment Counts

```
UPI: 69
Card: 44
Cash: 35
```

Explanation: groupby() groups transactions, sum() calculates totals, sort_values() ranks categories, and value_counts() counts transactions per payment method.

4.3 Sorting and Filtering

```
large_expenses = df[df["Amount"] >= 2000].sort_values("Amount", ascending=False)
print("Transactions of 2000 or more:", len(large_expenses))
print(large_expenses[["Date", "Category", "Amount"]].head())
```

Output

```
Transactions with Amount >= 2000: 18
Largest transactions:
      Date Category  Amount
2025-04-06 Shopping 2978.27
2025-02-19 Shopping 2822.81
2025-04-17 Shopping 2643.76
2025-01-17 Shopping 2597.00
2025-05-19 Shopping 2568.40
```

Explanation: filtering selects amounts of at least 2,000 and sorting displays the largest first, helping identify transactions that may merit review.

4.4 Correlation Analysis

```
df["Day"] = df["Date"].dt.day
df["Month_Number"] = df["Date"].dt.month
corr = df[["Amount", "Day", "Month_Number"]].corr(numeric_only=True)
print(corr.round(2))
```

Output: Correlation Matrix

```
              Amount   Day  Month_Number
Amount          1.00  0.15          0.03
Day             0.15  1.00         -0.20
Month_Number    0.03 -0.20          1.00
```

Explanation: correlation ranges from -1 to +1. Values near zero indicate weak linear relationships. Day and Month_Number are derived from dates, so the values are descriptive and do not prove cause and effect.

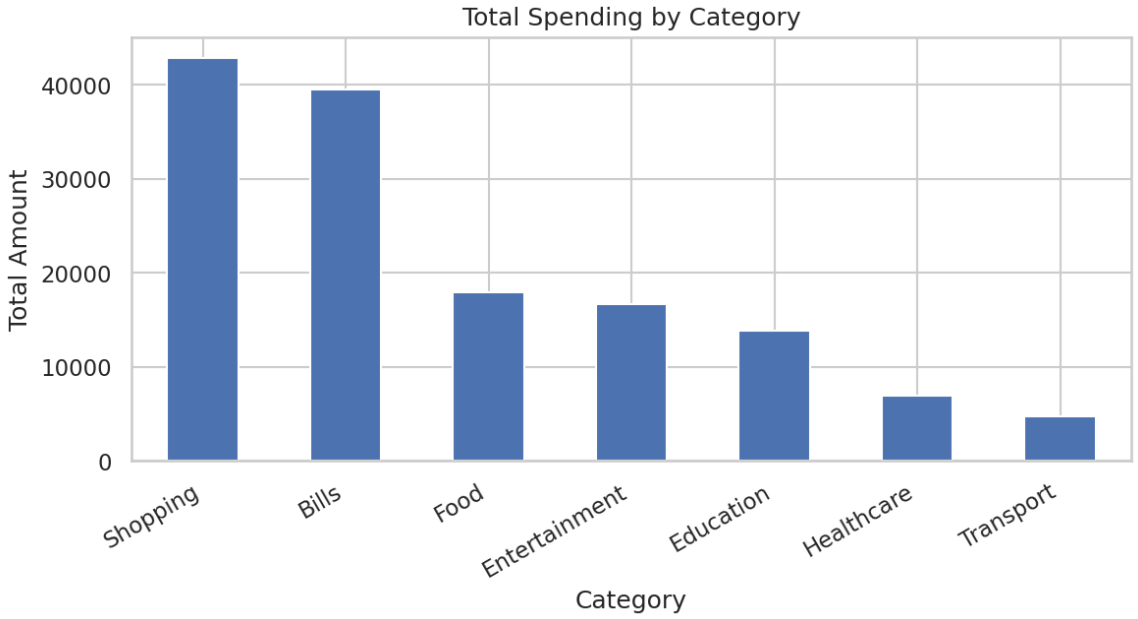
5. Data Visualization

Each code sample below is followed directly by its generated chart and a short interpretation.

5.1 Bar Chart — Spending by Category

```
category_totals = df.groupby("Category")["Amount"].sum().sort_values(ascending=False)
category_totals.plot(kind="bar", title="Total Spending by Category")
plt.xlabel("Category"); plt.ylabel("Total Amount")
plt.xticks(rotation=30, ha="right"); plt.tight_layout(); plt.show()
```

Output / Chart

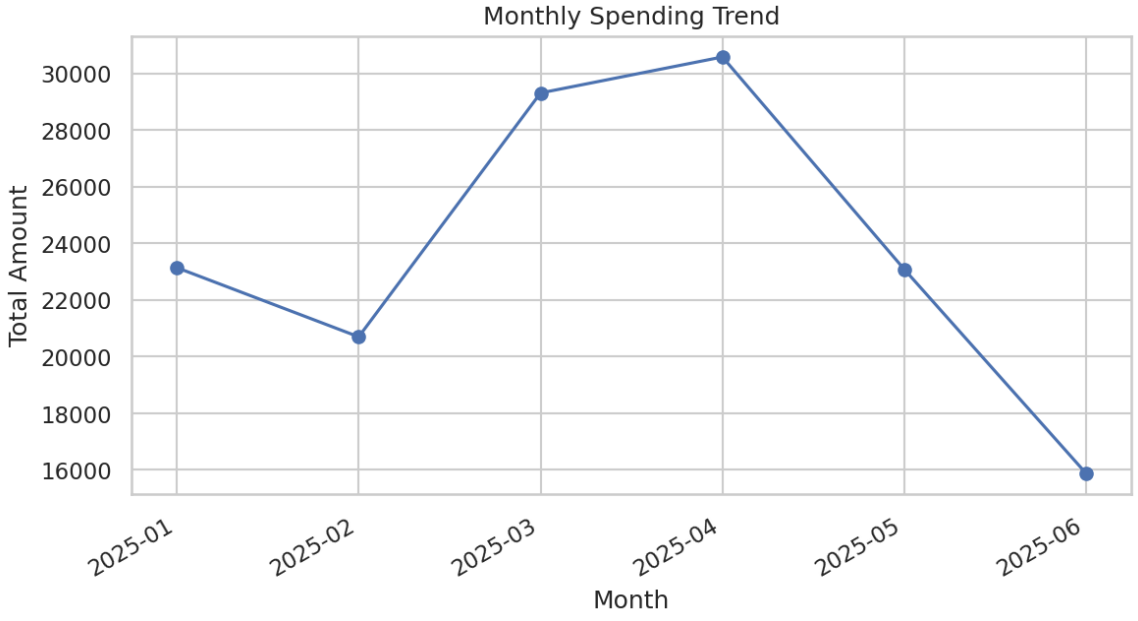


Interpretation: Shopping has the largest category total (42,893.49).

5.2 Line Chart — Monthly Spending

```
monthly_totals = df.groupby("Month")["Amount"].sum().sort_index()
monthly_totals.plot(kind="line", marker="o", title="Monthly Spending Trend")
plt.xlabel("Month"); plt.ylabel("Total Amount")
plt.xticks(rotation=30, ha="right"); plt.tight_layout(); plt.show()
```

Output / Chart

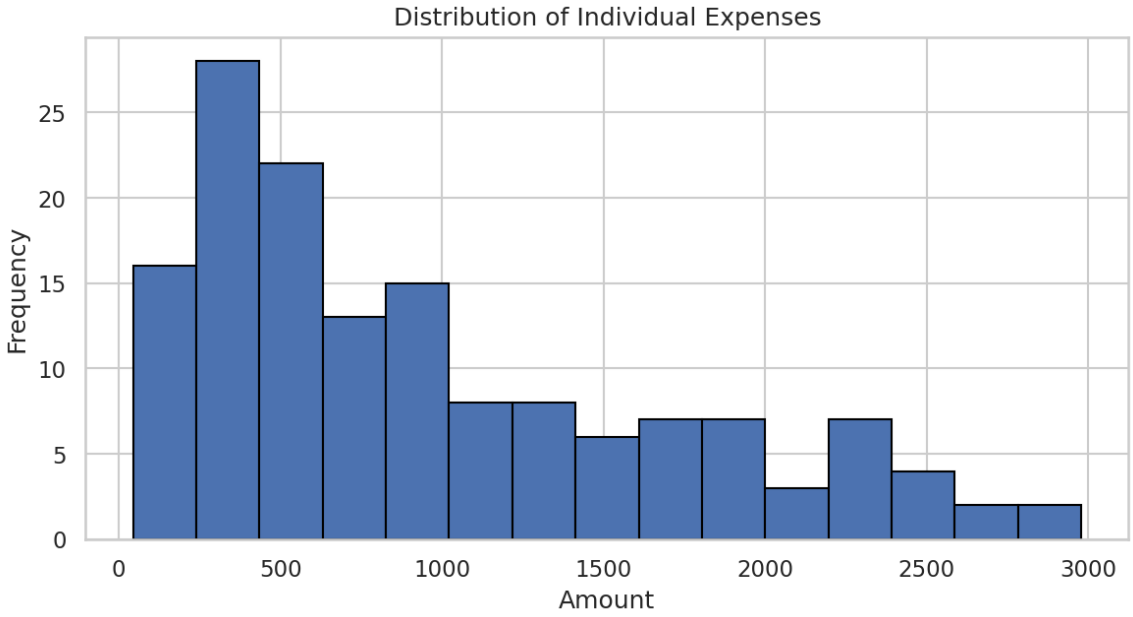


Interpretation: Spending peaks in 2025-04 (30,579.78) and is lowest in 2025-06 (15,873.09).

5.3 Histogram — Expense Distribution

```
df["Amount"].plot(kind="hist", bins=15, edgecolor="black",
                  title="Distribution of Individual Expenses")
plt.xlabel("Amount"); plt.ylabel("Frequency"); plt.tight_layout(); plt.show()
```

Output / Chart

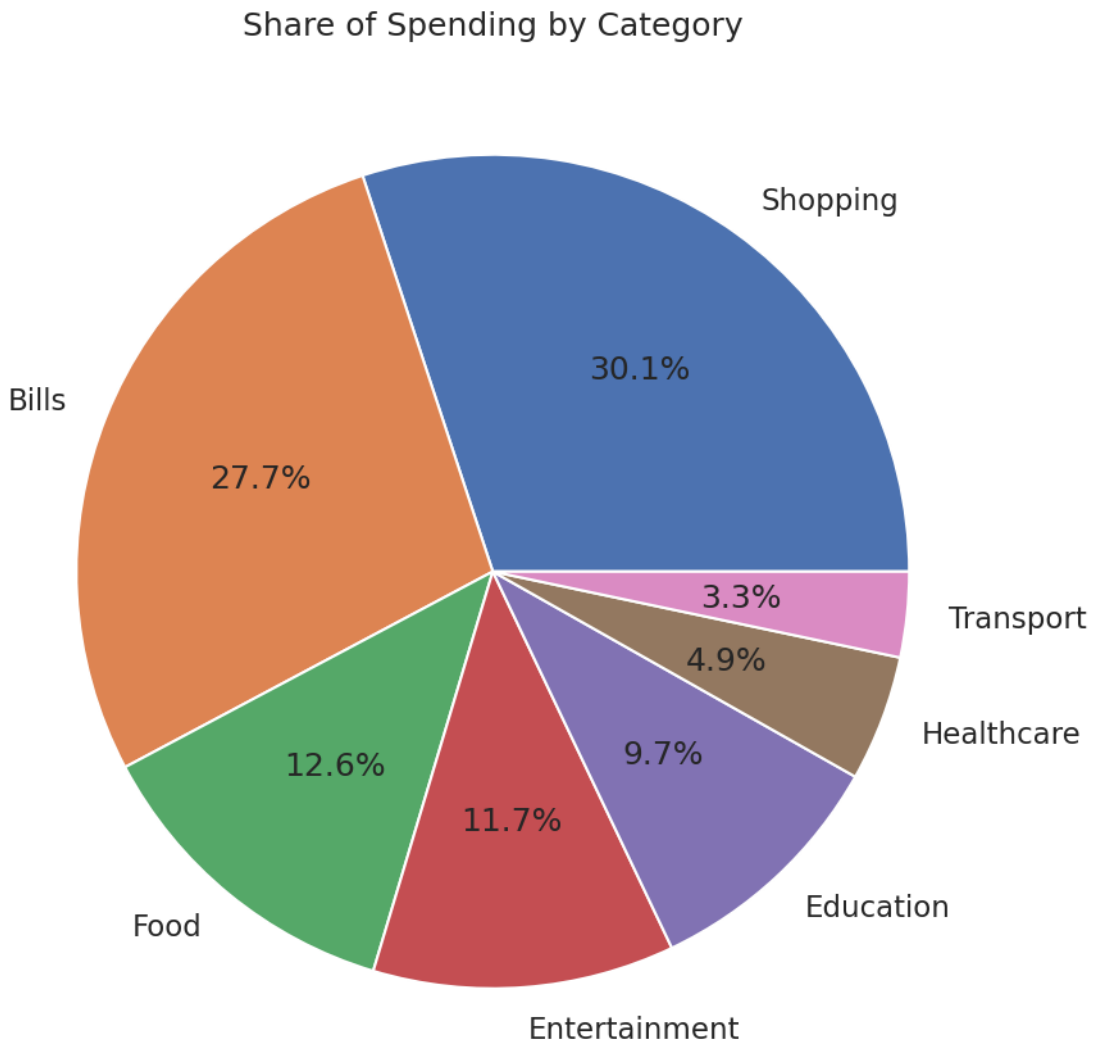


Interpretation: The histogram shows how often transactions occur within different amount ranges.

5.4 Pie Chart — Spending Share

```
category_totals.plot(kind="pie", autopct="%1.1f%%",
                     title="Share of Spending by Category")
plt.ylabel(""); plt.tight_layout(); plt.show()
```

Output / Chart

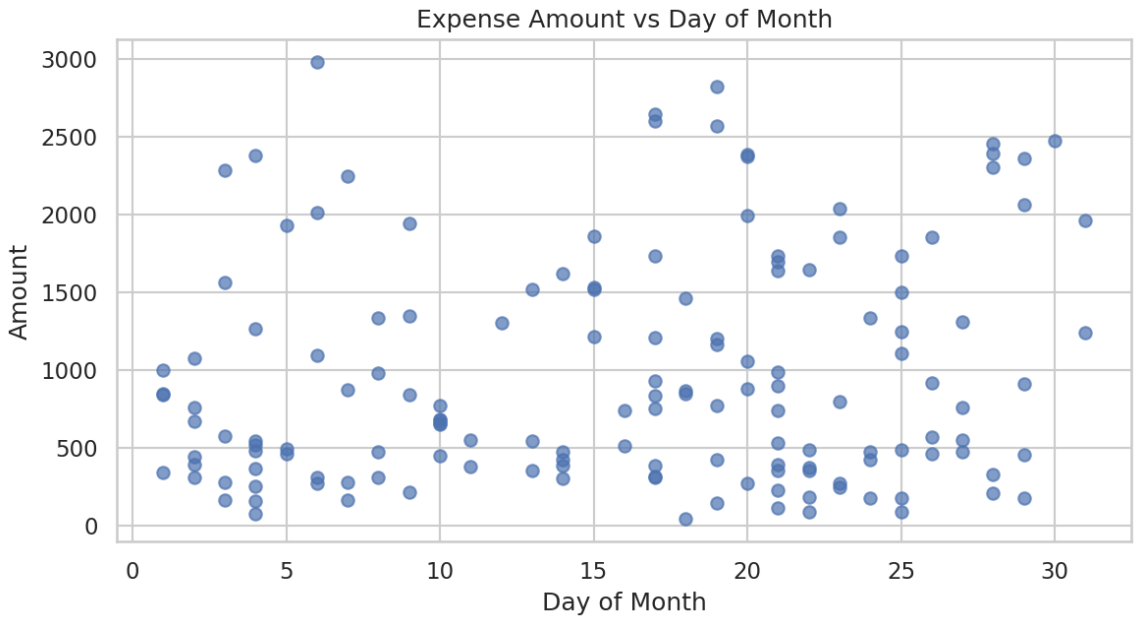


Interpretation: The pie chart compares each category's share of total recorded spending.

5.5 Scatter Plot — Amount vs Day

```
plt.scatter(df["Date"].dt.day, df["Amount"], alpha=0.7)
plt.title("Expense Amount vs Day of Month")
plt.xlabel("Day of Month"); plt.ylabel("Expense Amount")
plt.tight_layout(); plt.show()
```

Output / Chart

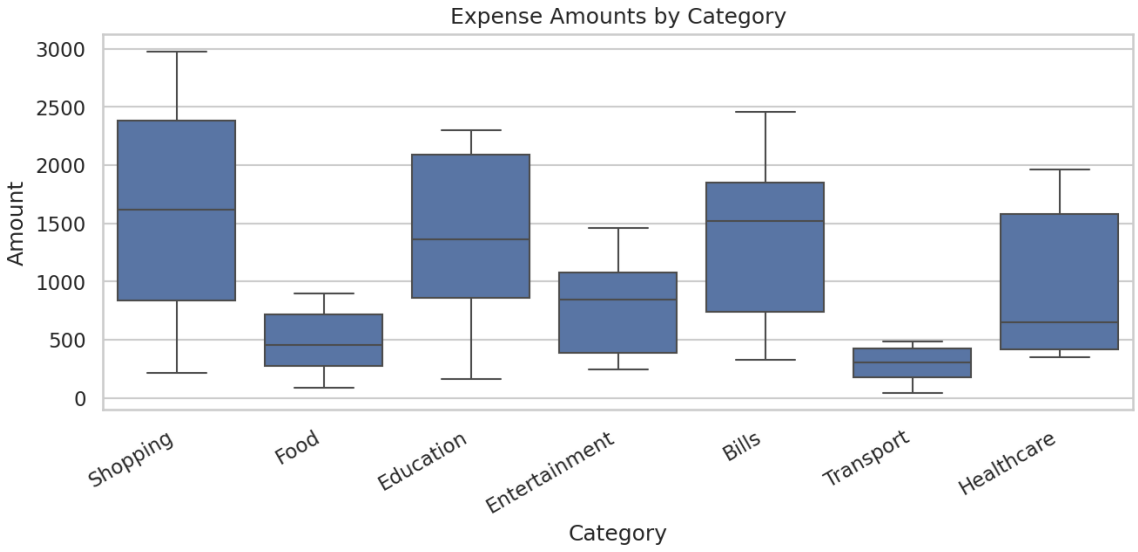


Interpretation: The scatter plot checks whether amounts appear to vary consistently by day of the month.

5.6 Seaborn Box Plot

```
sns.boxplot(data=df, x="Category", y="Amount")
plt.title("Expense Amounts by Category")
plt.xticks(rotation=30, ha="right"); plt.tight_layout(); plt.show()
```

Output / Chart

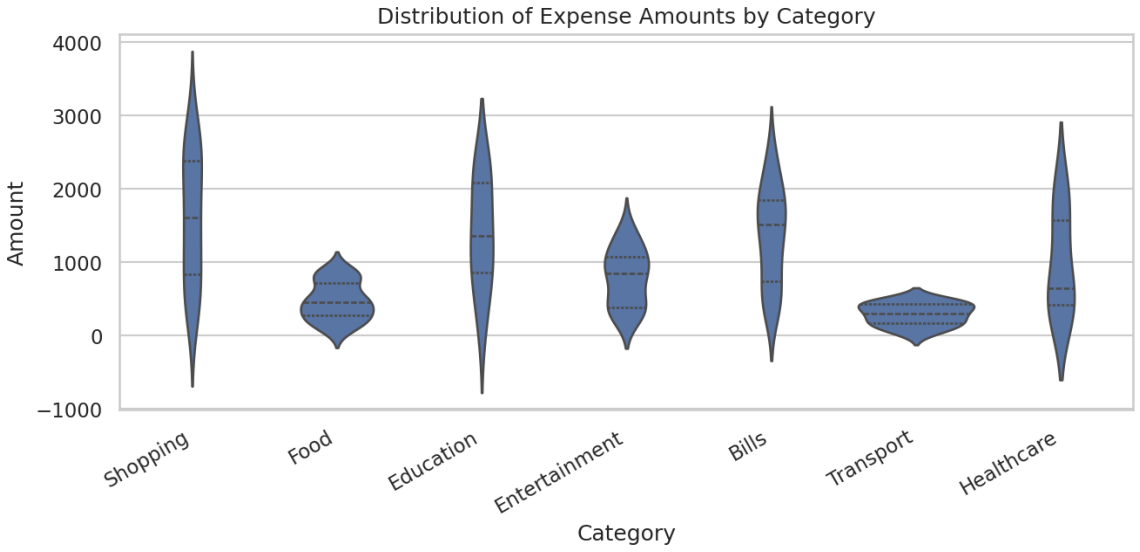


Interpretation: The box plot compares medians, spread and potential outliers across categories.

5.7 Seaborn Violin Plot

```
sns.violinplot(data=df, x="Category", y="Amount", inner="quartile")
plt.title("Distribution of Expense Amounts by Category")
plt.xticks(rotation=30, ha="right"); plt.tight_layout(); plt.show()
```

Output / Chart

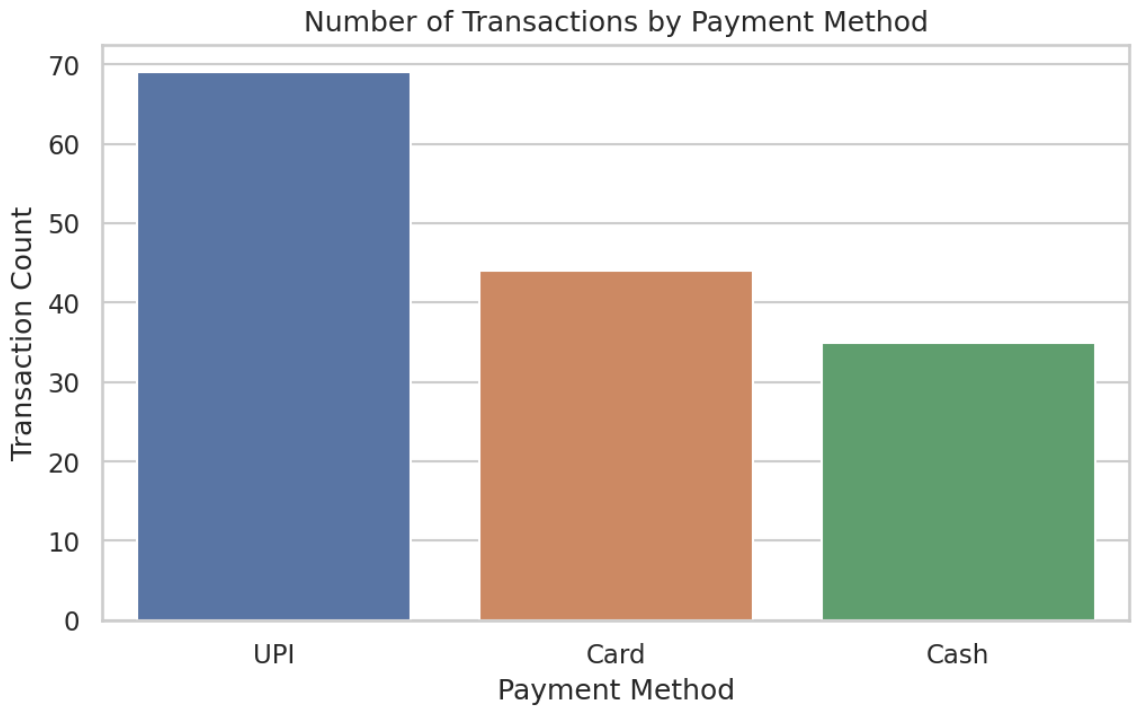


Interpretation: The violin plot shows the shape and density of amounts within each category.

5.8 Seaborn Count Plot

```
sns.countplot(data=df, x="Payment_Method", hue="Payment_Method", legend=False)
plt.title("Number of Transactions by Payment Method")
plt.xlabel("Payment Method"); plt.ylabel("Transaction Count")
plt.tight_layout(); plt.show()
```

Output / Chart

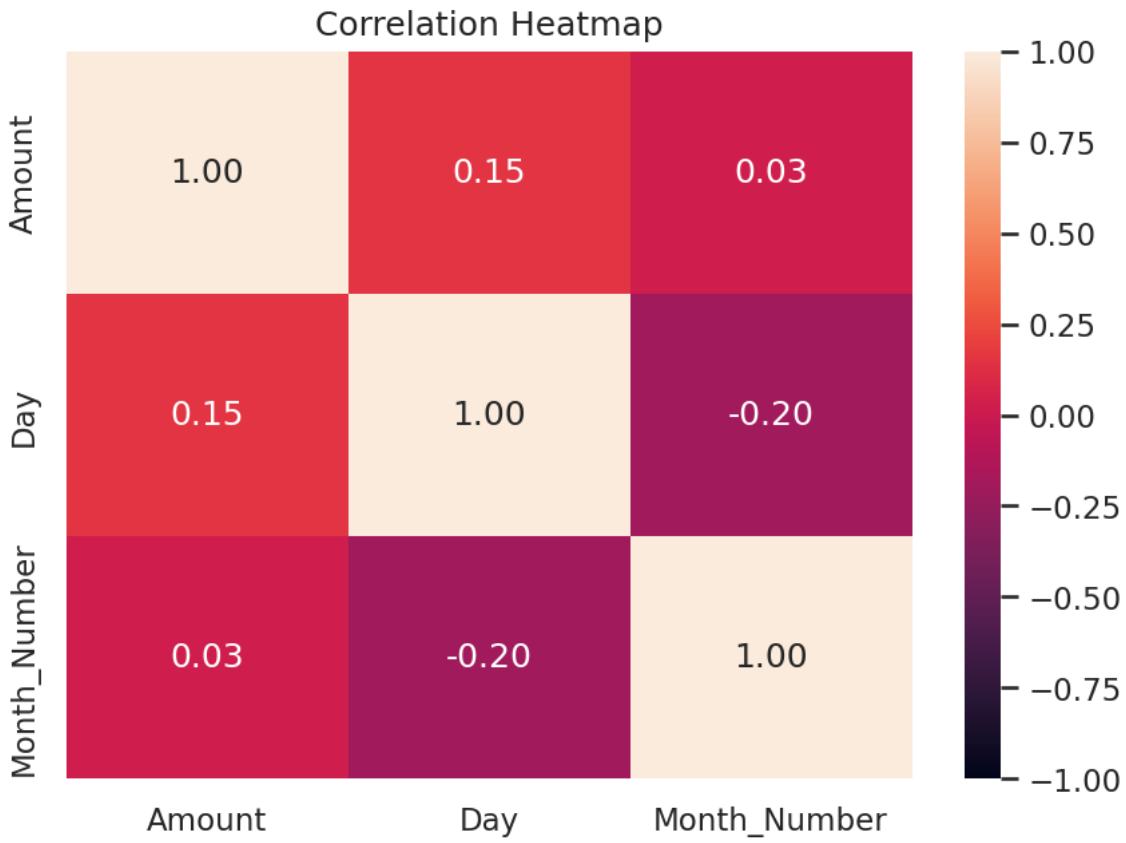


Interpretation: UPI is most frequent, with 69 transactions.

5.9 Seaborn Heatmap

```
corr = df[["Amount", "Day", "Month_Number"]].corr(numeric_only=True)
sns.heatmap(corr, annot=True, vmin=-1, vmax=1, fmt=".2f")
plt.title("Correlation Heatmap"); plt.tight_layout(); plt.show()
```

Output / Chart

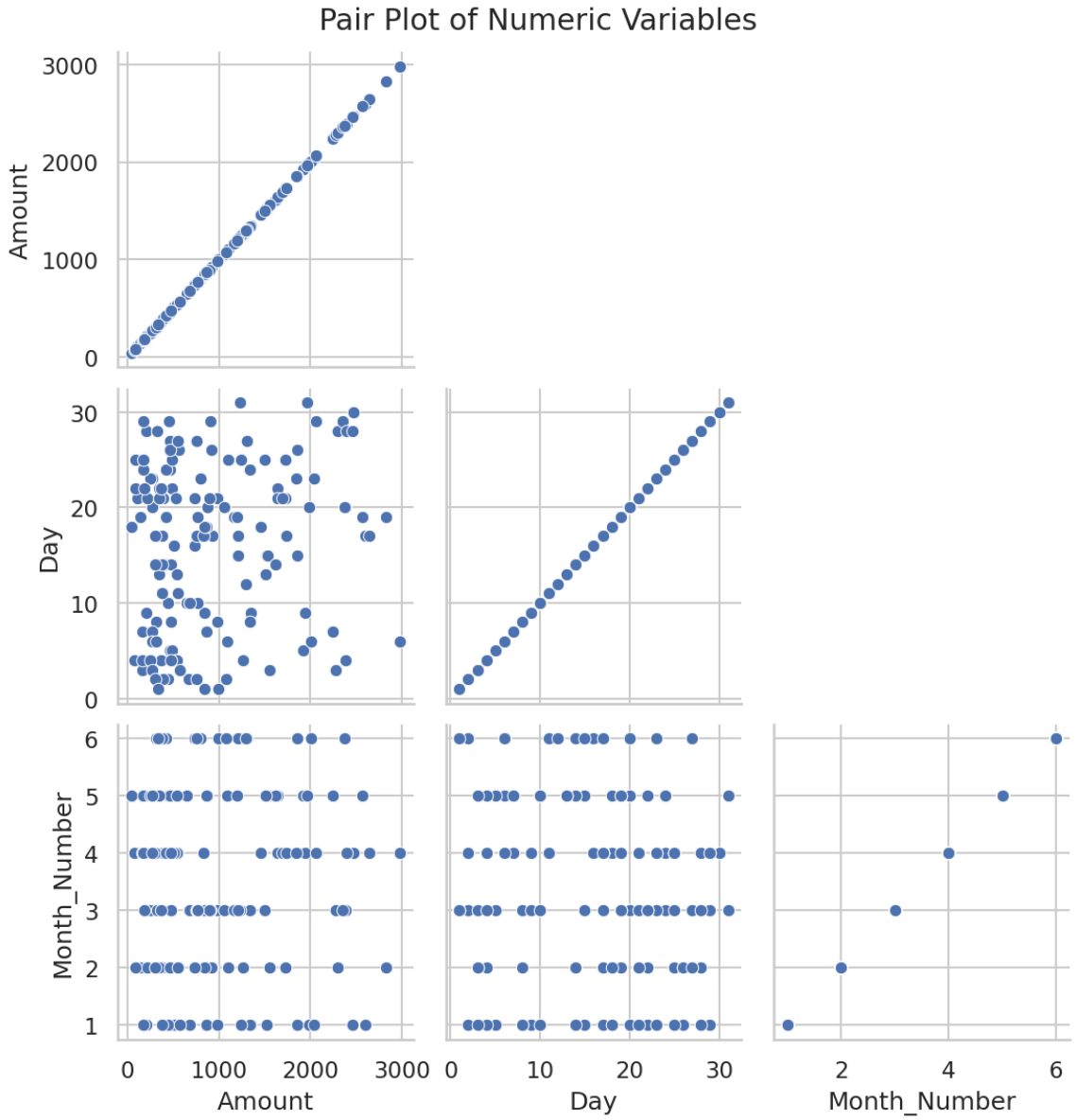


Interpretation: The heatmap gives a visual representation of the correlation matrix.

5.10 Seaborn Pair Plot

```
sns.pairplot(df[["Amount", "Day", "Month_Number"]], corner=True, diag_kind=None)
plt.show()
```

Output / Chart



Interpretation: The pair plot compares relationships between the selected numeric variables.

6. Findings

- The cleaned dataset contains 148 transactions from January to June 2025.
- Total recorded spending is 142,680.56.
- The mean transaction amount is 964.06, while the median is 755.28.
- Shopping is the highest-spending category, totalling 42,893.49.
- 2025-04 is the highest-spending month, totalling 30,579.78.
- UPI is the most frequent payment method, used in 69 transactions.
- The largest individual transaction is 2,978.27.
- The mean is above the median, indicating that larger transactions increase the average.

7. Conclusion

This project shows how Python can convert transaction-level data into useful spending summaries. Shopping is the largest spending category in this sample, and 2025-04 has the highest monthly total. Statistics describe typical transactions, grouping

identifies category and month totals, and charts make patterns easier to compare. Data cleaning is essential because inconsistent or invalid records can distort results. Since the dataset is synthetic, these findings are illustrative only.

Limitations and Future Improvements

- The dataset is synthetic and covers only six months.
- No income or budget field is included, so savings and budget overruns cannot be measured.
- Future work could add income, budgets, more months of records and an interactive dashboard.

Tools Used

- Pandas — data loading, cleaning, grouping, sorting and filtering.
- NumPy — numerical statistics.
- Matplotlib — bar, line, histogram, pie and scatter charts.
- Seaborn — box, violin, count, heatmap and pair plots.